

SECURE MESSAGING APPLICATION

Security Engineering Report

Technical report covering the application design, implemented security controls, and validation results.

Project type	Secure software engineering / application security
Core stack	Python, Flask, SQLite, SQLAlchemy
Cryptography	AES-256-GCM, RSA-2048 OAEP-SHA256, Fernet
Security controls	bcrypt password hashing, CSRF protection, authenticated routes
Environment	Parrot OS virtual machine - local development lab
Report date	August 2026

Scope note

This report covers a local security-focused prototype. The application is not intended for production deployment and does not provide client-side end-to-end encryption.

Project Summary

The Secure Messaging Application is a Flask-based prototype that supports account registration, login, encrypted message delivery, and recipient-only inbox access. It combines authentication, hybrid cryptography, protected key storage, database persistence, and standard web-security controls.

Implemented controls

- User registration and login with bcrypt password hashing.
- Per-message encryption using a fresh AES-256-GCM key and nonce.
- RSA-2048 OAEP with SHA-256 to protect each AES message key.
- Encrypted RSA private-key storage using Fernet and an environment-based application key.
- Ciphertext, nonce, and wrapped key stored in SQLite instead of a plaintext message field.
- CSRF protection, authenticated chat access, generic login errors, and debug mode disabled.
- Security validation with direct tests and a Bandit static-analysis scan.

Technology used

Area	Implementation
Web application	Flask 3.1.3, Flask-Login, Flask-WTF
Data layer	Flask-SQLAlchemy / SQLAlchemy 2.0.52 / SQLite
Password security	Flask-Bcrypt / bcrypt
Message encryption	AES-256-GCM via cryptography
Key protection	RSA-2048 OAEP-SHA256 + Fernet
Validation	Bandit 1.9.4 and manual negative tests

Message processing

Before storage, each message is encrypted with a new AES-256-GCM key. That key is wrapped with the recipient's RSA public key. When the recipient opens the inbox, the application unwraps the AES key and decrypts the message for display.

Application Structure and Interface

The application is separated into models, routes, templates, cryptographic utilities, configuration, and startup files. The registration interface is shown below.

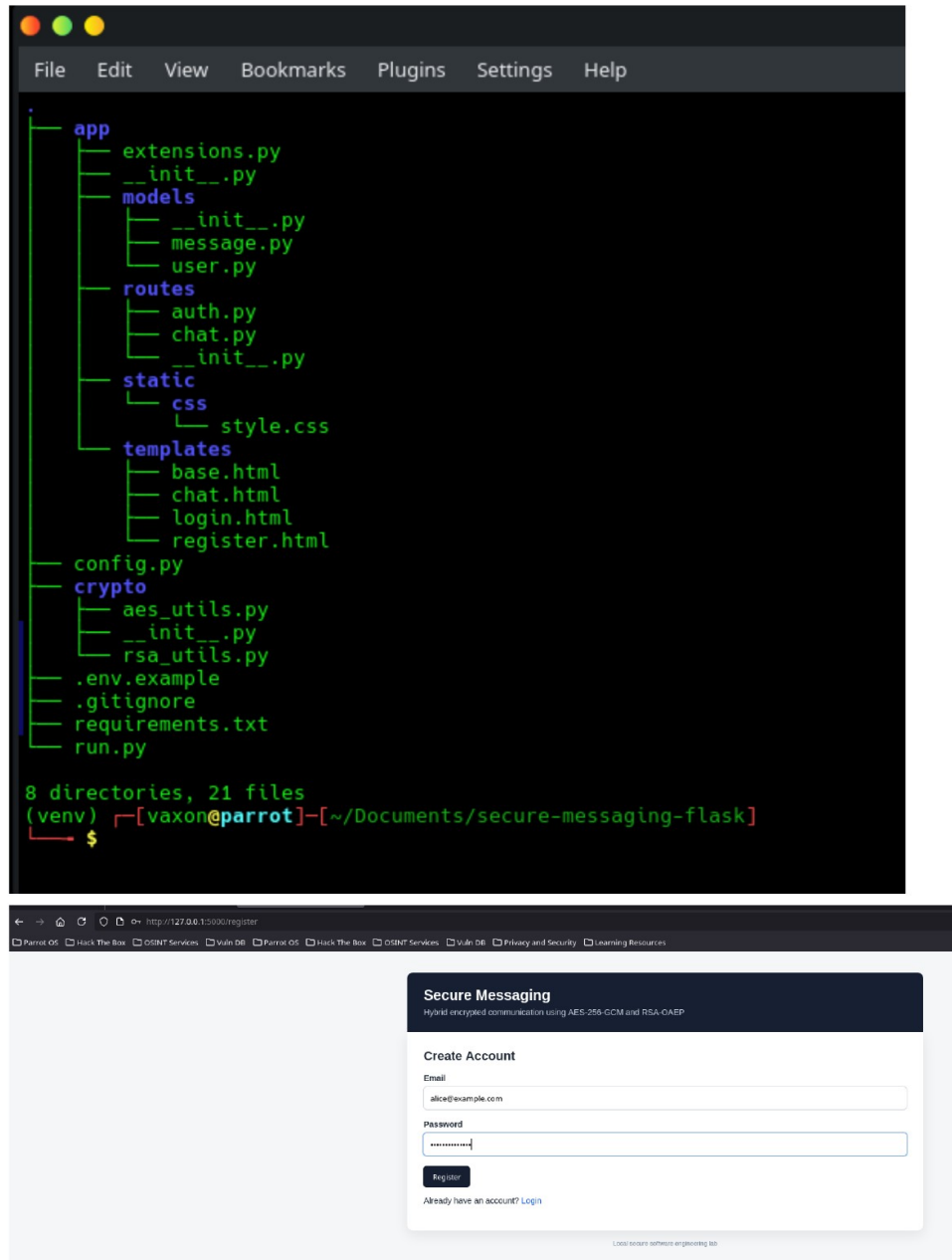


Figure: application structure and account-registration interface.

Registration, Login and Secure Chat

Account creation, login, and the authenticated chat screen were tested in the local Flask environment. The chat page identifies the signed-in user and provides message composition plus an encrypted inbox.

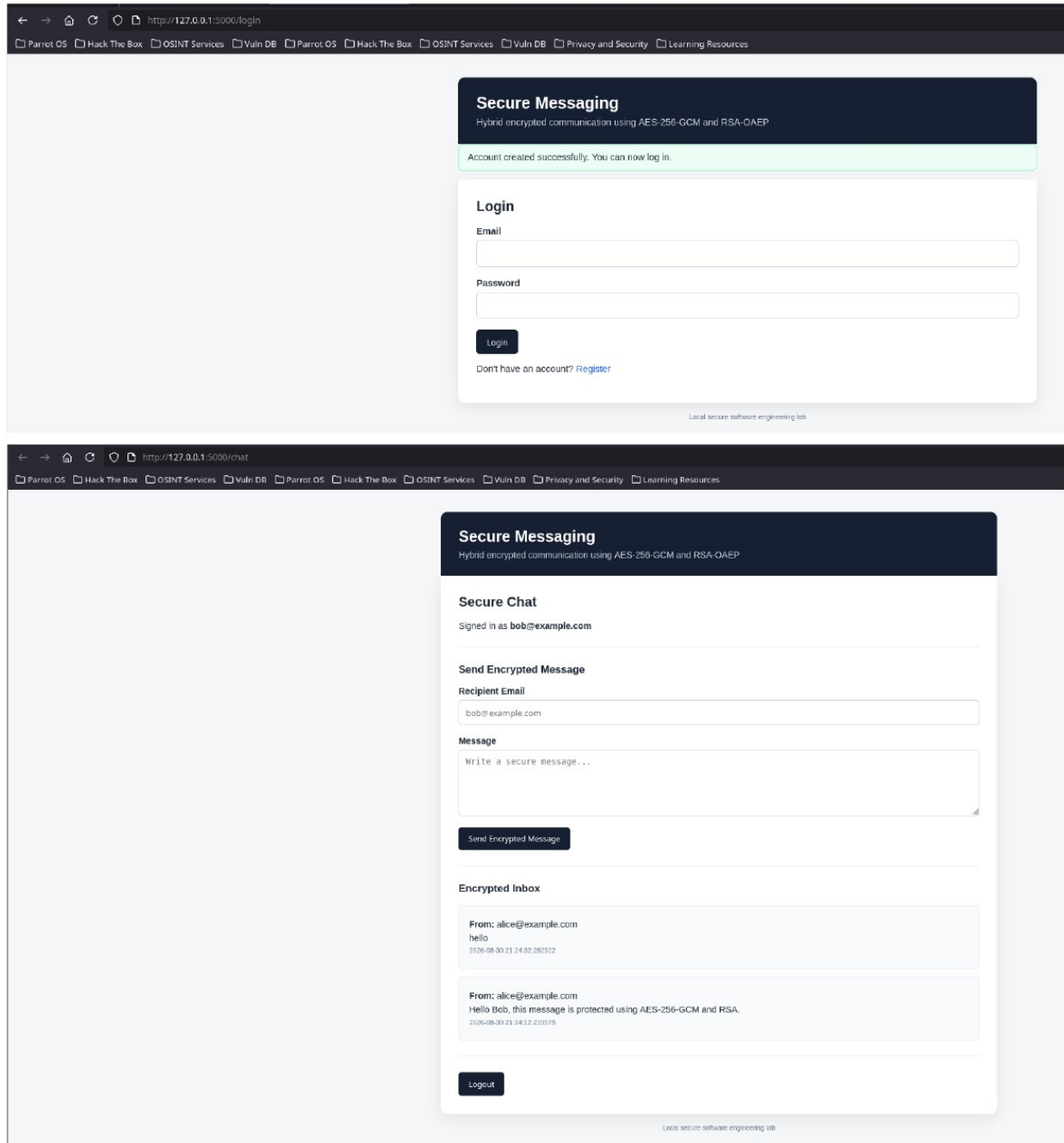


Figure: account creation and authenticated chat interface.

Encrypted Messaging and Database Storage

Messages are encrypted before database persistence. The message model stores a wrapped AES key, nonce, ciphertext, sender, recipient, and timestamp; it does not define a plaintext message column.

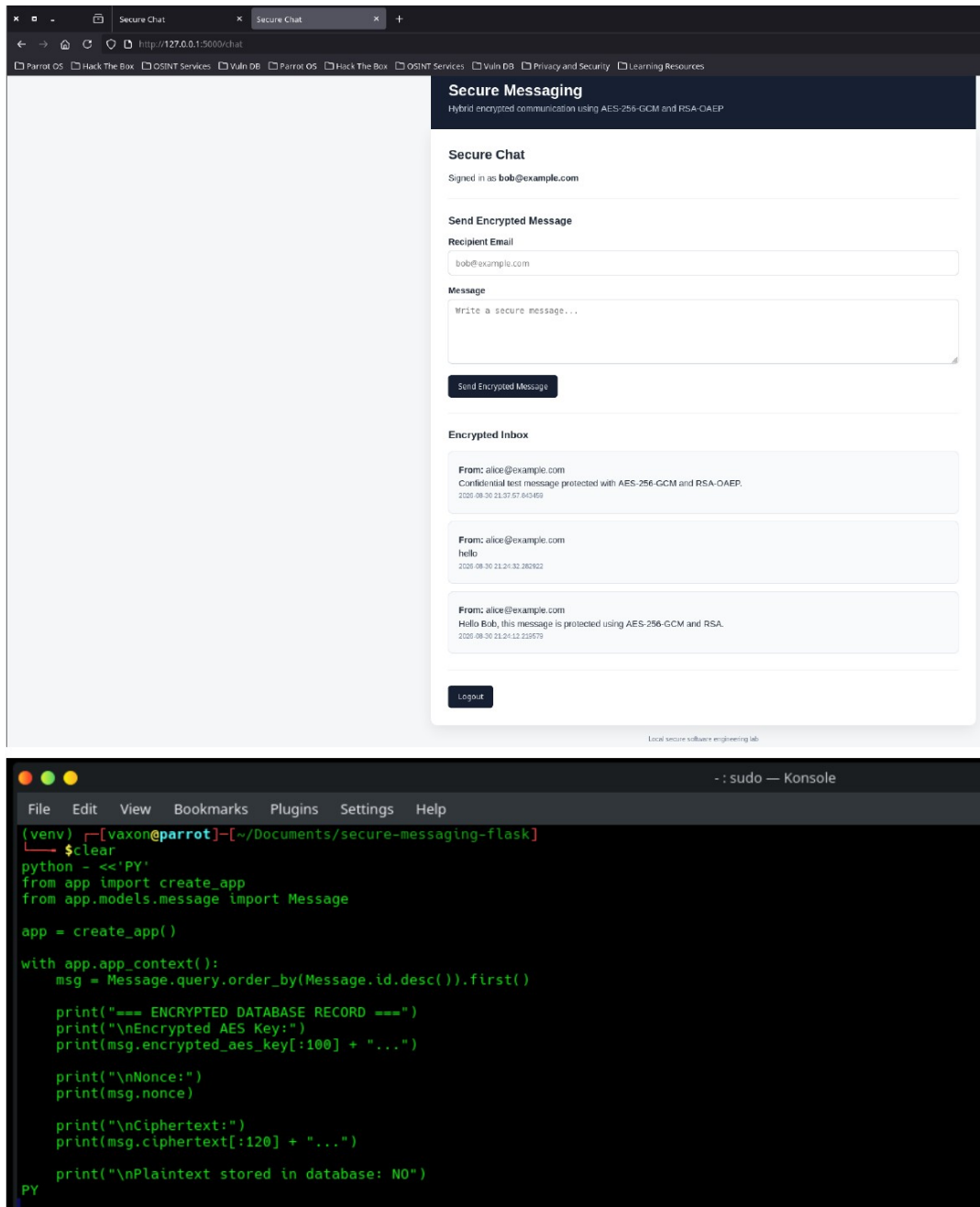
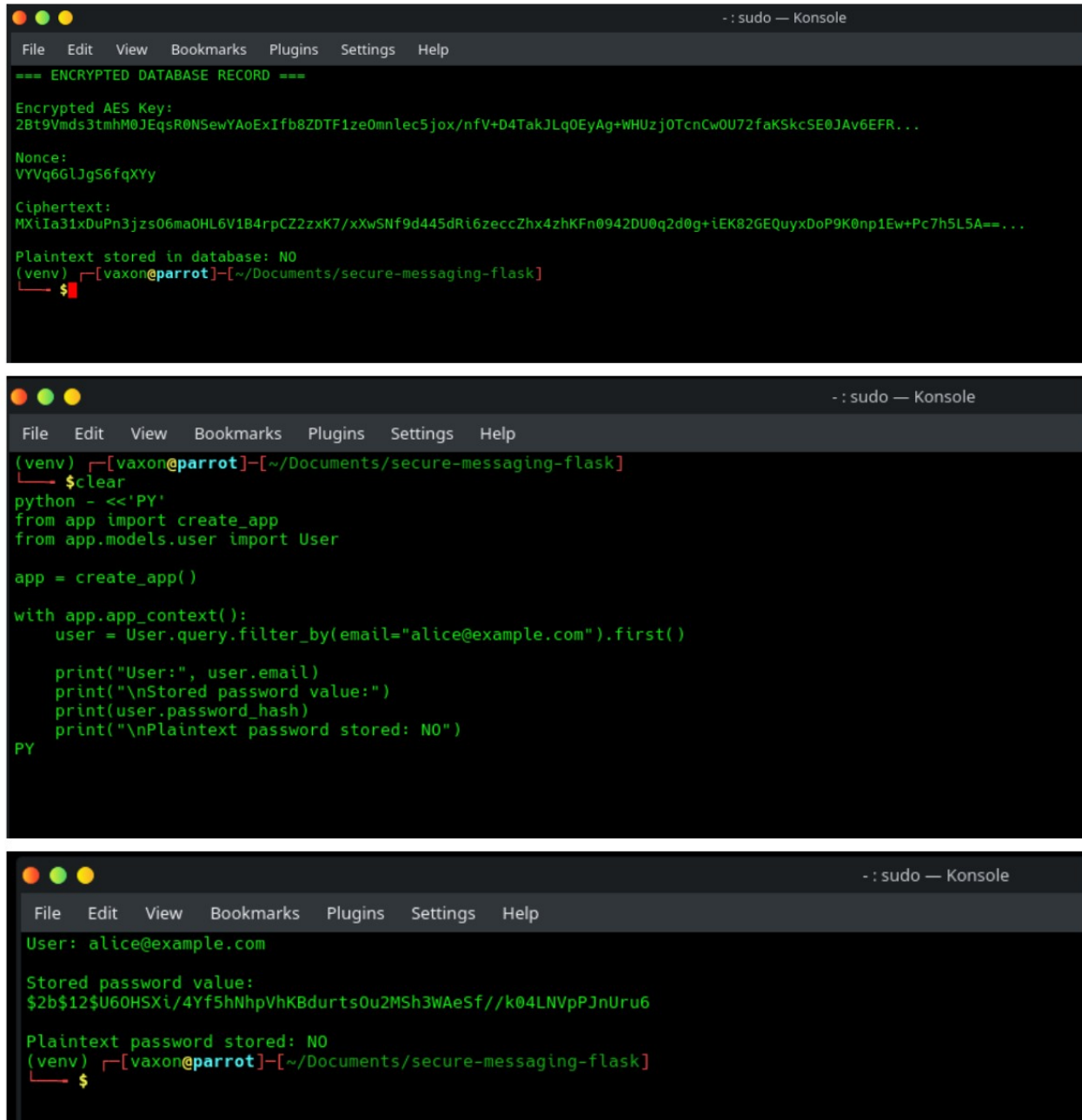


Figure: recipient inbox and database inspection of an encrypted message record.

Ciphertext at Rest and Password Hashing

The captured database inspection shows encrypted message material rather than the readable message. A separate user-record check shows a bcrypt hash instead of the original password.



```

- : sudo — Konsole
File Edit View Bookmarks Plugins Settings Help
=== ENCRYPTED DATABASE RECORD ===

Encrypted AES Key:
2Bt9Vmds3tmhM0JEqsR0NSewYAOExIfb8ZDTF1ze0mnlec5jox/nfV+D4TakJLq0EyAg+WHUzj0TcnCw0U72faKSkcSE0JAv6EFR...

Nonce:
VYVq6GLJgS6fqXYy

Ciphertext:
MXiIa31xDuPn3jzs06ma0HL6V1B4rpCZ2zxK7/xXwSNf9d445dRi6zeccZhx4zhKFn0942DU0q2d0g+iEK82GEQuyxDoP9K0np1Ew+Pc7h5L5A==...

Plaintext stored in database: NO
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask
$

- : sudo — Konsole
File Edit View Bookmarks Plugins Settings Help
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask
$clear
python - <<'PY'
from app import create_app
from app.models.user import User

app = create_app()

with app.app_context():
    user = User.query.filter_by(email="alice@example.com").first()

    print("User:", user.email)
    print("\nStored password value:")
    print(user.password_hash)
    print("\nPlaintext password stored: NO")
PY

- : sudo — Konsole
File Edit View Bookmarks Plugins Settings Help
User: alice@example.com

Stored password value:
$2b$12$U60HSXi/4Yf5hNhpVhKBdurts0u2MSh3WAeSf//k04LNVpPJnUru6

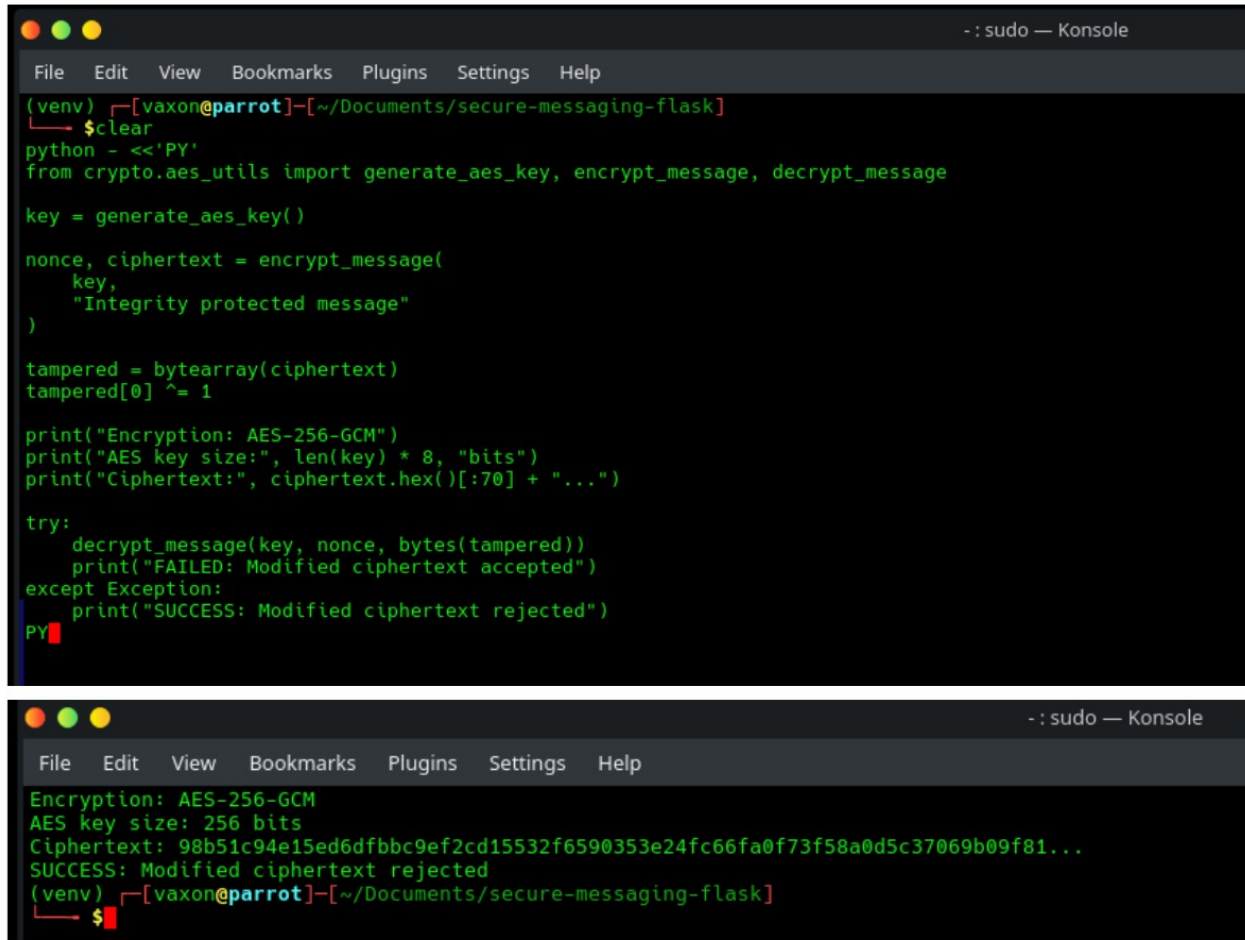
Plaintext password stored: NO
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask
$

```

Figure: encrypted database fields and bcrypt password-hash verification.

AES-GCM Tamper Detection

A message was encrypted with AES-256-GCM, one ciphertext byte was modified, and decryption was attempted. The modified ciphertext was rejected, confirming that integrity protection is active in the tested workflow.



```
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask
$clear
python - <<'PY'
from crypto.aes_utils import generate_aes_key, encrypt_message, decrypt_message

key = generate_aes_key()

nonce, ciphertext = encrypt_message(
    key,
    "Integrity protected message"
)

tampered = bytearray(ciphertext)
tampered[0] ^= 1

print("Encryption: AES-256-GCM")
print("AES key size:", len(key) * 8, "bits")
print("Ciphertext:", ciphertext.hex()[:70] + "...")

try:
    decrypt_message(key, nonce, bytes(tampered))
    print("FAILED: Modified ciphertext accepted")
except Exception:
    print("SUCCESS: Modified ciphertext rejected")
PY
```

```
Encryption: AES-256-GCM
AES key size: 256 bits
Ciphertext: 98b51c94e15ed6dfbbc9ef2cd15532f6590353e24fc66fa0f73f58a0d5c37069b09f81...
SUCCESS: Modified ciphertext rejected
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask
$
```

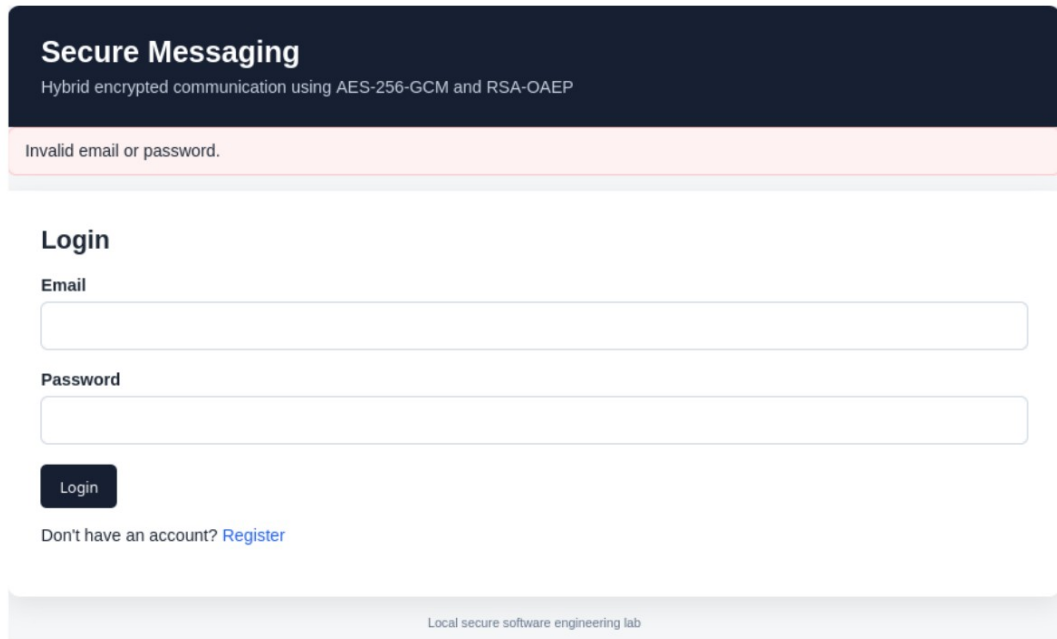
Figure: AES-256-GCM tamper test showing rejection of modified ciphertext.

Implementation note

The application generates a new 256-bit AES key and 12-byte nonce for each message. RSA-OAEP with SHA-256 protects the AES key for the recipient.

Authentication and Protected Routes

Invalid credentials return a generic error message. Direct access to the chat page without a valid login is redirected to the login screen.



The screenshot shows the 'Secure Messaging' login interface. At the top, a dark blue header contains the title 'Secure Messaging' and the text 'Hybrid encrypted communication using AES-256-GCM and RSA-OAEP'. Below this, a light pink error message states 'Invalid email or password.' The main form area is white and contains a 'Login' section with two input fields labeled 'Email' and 'Password', a 'Login' button, and a link 'Don't have an account? Register'. The footer of the form area says 'Local secure software engineering lab'.

Figure: generic response returned for invalid credentials.

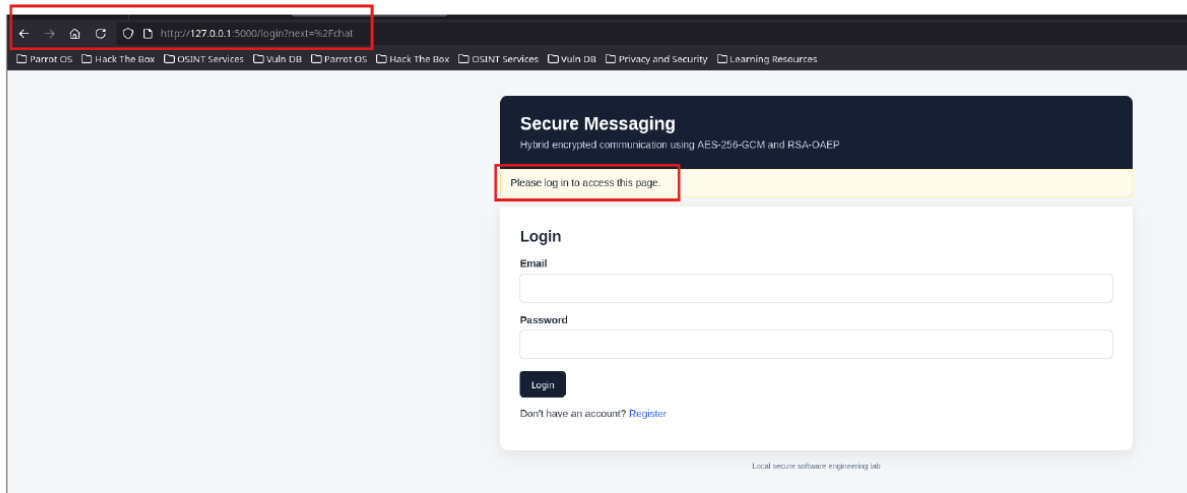


Figure: unauthenticated request to /chat redirected to the login page.

Static Security Scan

Bandit was run against the application, cryptography modules, run.py, and config.py. The captured scan reported no identified issues across 434 lines of Python code.

```
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask
└─$ bandit -r app crypto run.py config.py
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.13.5
Run started:2026-08-30 21:51:25.827846+00:00

Test results:
    No issues identified.

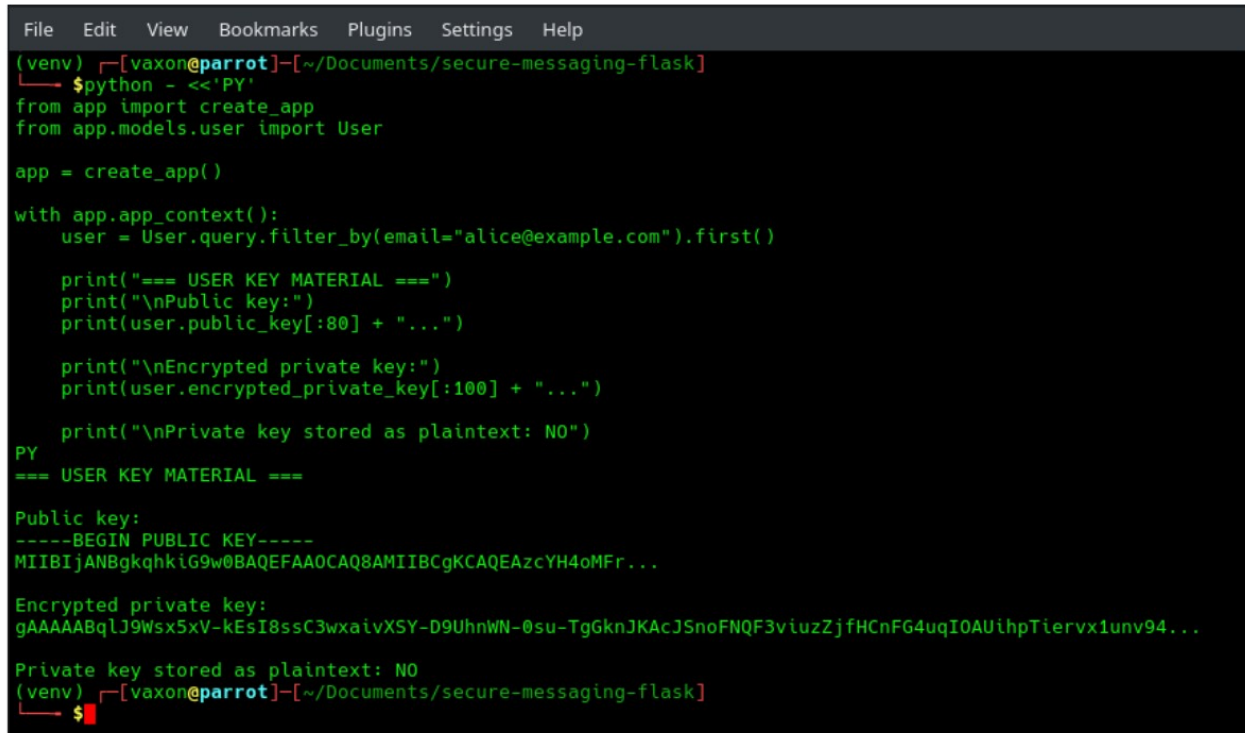
Code scanned:
    Total lines of code: 434
    Total lines skipped (#nosec): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask
└─$
```

Figure: Bandit 1.9.4 scan result with no issues identified in the captured run.

RSA Key Protection

Each account receives an RSA-2048 key pair during registration. The public key is stored for message-key encryption, while the private key is stored as a Fernet-encrypted value rather than plaintext PEM data.



```

File Edit View Bookmarks Plugins Settings Help
(venv) [vaxon@parrot] ~/Documents/secure-messaging-flask
$python - <<'PY'
from app import create_app
from app.models.user import User

app = create_app()

with app.app_context():
    user = User.query.filter_by(email="alice@example.com").first()

    print("=== USER KEY MATERIAL ===")
    print("\nPublic key:")
    print(user.public_key[:80] + "...")

    print("\nEncrypted private key:")
    print(user.encrypted_private_key[:100] + "...")

    print("\nPrivate key stored as plaintext: NO")
PY
=== USER KEY MATERIAL ===

Public key:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAzcyH4oMFr...

Encrypted private key:
gAAAAABqIJ9Wsx5xV-kEsI8ssC3wxaivXSY-D9UhnWN-0su-TgGknJKAcJSnoFNQF3viuzZjfHCnFG4uqIOAUihpTervx1unv94...

Private key stored as plaintext: NO
(venv) [vaxon@parrot] ~/Documents/secure-messaging-flask
$

```

Figure: stored public key and encrypted private-key value from the user record.

Design boundary

The application server holds the environment key used to decrypt stored user private keys. The design therefore provides server-managed hybrid encryption rather than client-side end-to-end encryption.

Encryption Randomness and CSRF Protection

Two encryptions of the same plaintext produced different ciphertext values. A POST request to the login endpoint without a CSRF token returned HTTP 400 with “The CSRF token is missing.”

The figure consists of two terminal screenshots. The top screenshot shows a Python script in a terminal window titled '(venv) [vaxon@parrot]~/Documents/secure-messaging-flask'. The script generates two different AES keys, encrypts the same message 'Same confidential message' with each key, and prints the resulting ciphertexts in hexadecimal. The ciphertexts are different, demonstrating encryption randomness. The bottom screenshot shows a terminal window titled 'project: bash — Konsole' with a curl command being executed: 'curl -i -X POST http://127.0.0.1:5000/login -d "email=alice@example.com&password=test"'. The output shows an HTTP 400 status code and a message: 'The CSRF token is missing.'

```
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask
$python - <<'PY'
from crypto.aes_utils import generate_aes_key, encrypt_message

message = "Same confidential message"

key1 = generate_aes_key()
key2 = generate_aes_key()

nonce1, ciphertext1 = encrypt_message(key1, message)
nonce2, ciphertext2 = encrypt_message(key2, message)

print("Plaintext:")
print(message)

print("\nEncryption #1:")
print(ciphertext1.hex()[:80] + "...")

print("\nEncryption #2:")
print(ciphertext2.hex()[:80] + "...")

print("\nCiphertexts are different:", ciphertext1 != ciphertext2)
PY
Plaintext:
Same confidential message

Encryption #1:
26ab2ebf367dcedd7fcd82add71bea025194a47100e788d4b5b0f7282cae2617fb5c48f5795da997...

Encryption #2:
ade4b4fe9a77556ba8726d1e51ca41a8eb94502555322de0b30174cbb1ae6ca548691a5a716f55...

Ciphertexts are different: True
(venv) [vaxon@parrot]~/Documents/secure-messaging-flask

project: bash — Konsole
[vaxon@parrot]~/Documents/project
$curl -i -X POST http://127.0.0.1:5000/login \
-d "email=alice@example.com&password=test"
HTTP/1.1 400 Bad Request
Server: Werkzeug/3.1.3 Python/3.11.5
Date: Sun, 30 Aug 2025 21:54:09 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 117
Vary: Cookie
Connection: close

<!doctype html>
<html lang=en>
<title>400 Bad Request</title>
<h1>Bad Request</h1>
<p>The CSRF token is missing.</p>
[vaxon@parrot]~/Documents/project
```

Figure: repeated encryption produces different ciphertext, and a tokenless POST request is rejected by CSRF protection.

Implementation Review

The source archive was reviewed against the recorded test evidence. Security-sensitive functions are separated into dedicated authentication, messaging, and cryptographic modules.

Component	What it does
app/routes/auth.py	Registration, login, logout and account checks
app/routes/chat.py	Recipient lookup, message encryption, persistence and inbox decryption
crypto/aes_utils.py	AES-256-GCM key generation, encryption and decryption
crypto/rsa_utils.py	RSA key generation, OAEP key wrapping and Fernet private-key protection
app/models/user.py	Email, bcrypt hash, public key and encrypted private key
app/models/message.py	Sender, recipient, wrapped AES key, nonce and ciphertext
config.py	Loads application secrets from .env and configures SQLite/cookie settings
run.py	Starts Flask locally on 127.0.0.1:5000 with debug disabled

Implementation details confirmed in source code

- AESGCM.generate_key(bit_length=256) is used for message keys.
- RSA keys are 2048 bits and use OAEP with SHA-256.
- Passwords are generated and checked through Flask-Bcrypt.
- CSRFProtect is enabled globally.
- The chat route is protected with Flask-Login @login_required.
- The local server runs with debug=False.

RESULTS

Test Summary

Test	Result	Evidence
Account registration and login	PASS	Application screenshots
Encrypted message send / receive	PASS	Recipient inbox
Ciphertext stored instead of plaintext field	PASS	Database model + inspection
bcrypt password storage	PASS	User-record inspection
AES-GCM tamper rejection	PASS	Modified ciphertext test
Invalid login handling	PASS	Generic error response
Protected chat route	PASS	Redirect to login
Bandit static analysis	PASS	No issues identified
Encrypted private-key storage	PASS	User key inspection
Repeated plaintext produces different ciphertext	PASS	Randomness test
CSRF token enforcement	PASS	HTTP 400 without token

Overall result

The tested build completed the core functional and security checks recorded in this report. The results are supported by source review and captured test evidence.

LIMITATIONS

Limitations and Future Work

The application is a local prototype and is not intended to represent a production messaging service. The following items would require further work before deployment in a real environment.

Current limitation	Recommended next step
Server can access the private-key encryption key	Move key custody to clients or a dedicated KMS/HSM design
Local Flask development server	Deploy behind a production WSGI server and HTTPS
SQLite and <code>db.create_all()</code>	Use migrations and a production database for larger deployments
No MFA, rate limiting or account recovery	Add identity hardening and abuse controls
Limited security logging	Add structured audit and security-event logging

Repository preparation

- Keep `.env`, `instance/`, `venv/` and generated database files out of the repository.
- Replace the current `.env.example` values with obvious placeholders rather than usable-looking secrets.
- Include a README with setup instructions, dependencies, usage, and a link to this report.

Summary

The application integrates authentication, hybrid message encryption, encrypted key storage, protected routes, password hashing, CSRF enforcement, and static security analysis in a single Flask implementation. The recorded tests confirm the behavior described in this report.

CONCLUSION

Conclusion

The Secure Messaging Application combines authenticated access, encrypted message storage, protected cryptographic keys, and direct security testing in a working Flask prototype.

Key verified results

- The application runs and supports registration, login and messaging.
- Messages are encrypted with AES-256-GCM and the AES key is protected with RSA-OAEP.
- Passwords and RSA private keys are not stored in plaintext.
- Unauthorized requests and tokenless form submissions are blocked.
- Tampering with ciphertext causes decryption to fail.
- The captured Bandit scan reports no issues.

Report title: Secure Messaging Application - Security Engineering Report

End of report